

Market Basket Analysis & Smart Retail Recommendation System

Project Summary

This project applies market basket analysis to a real-world UK online retail dataset to uncover which products customers buy together, and turns those findings into a live, interactive recommendation dashboard that any business user can query without a single line of code.

At its core, the project uses the Apriori algorithm to mine association rules from 18,668 transactions. Those rules power a Streamlit web app that, when a product is selected, returns:

- The top 3 products most frequently bought alongside it ("Frequently Bought Together")
- The top 3 cross-category products that the same customers tend to buy ("Cross-Sell Opportunities")

Stages of the Project

Phase 1 – Data Exploration & Cleaning (Week 1)

The dataset 'OnlineRetail.csv' contained 541,909 rows representing individual line items across customer invoices. Before any modeling, I audited the data carefully:

- Identified 1,454 null descriptions – rows with no product name are useless for basket analysis, so they were dropped
- Normalized whitespace in product descriptions using regex substitution (`re.sub(r'\s+', ' ', x).strip()`) to prevent duplicate item groupings from formatting inconsistencies
- Removed canceled transactions – invoices prefixed with "C" represent returns and would introduce negative values into the model, so all were excluded
- Analyzed geographic distribution – the UK accounted for over 90% of all transactions, so the analysis was scoped to UK-only data to ensure purchasing patterns reflect a single market

After cleaning, the dataset contained 531,167 rows.

```
1 # sanity checks
2 print(df.isnull().sum())
3 print(df.describe())
```

✓ [3] 164ms

InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0

```
1 # cleaning
2 # #drop null descriptions
3 df.dropna(axis=0, subset=["Description"], inplace=True) #axis0 focus on rows, subset(area), inplace applies
  drop to dataset
4
5 ##remove trailing/leading spaces - not to confuse grouping, (s+- represent one or more spaces)
6 df['Description'] = df['Description'].astype('str').apply(lambda x: re.sub(r'\s+', ' ', x).strip())
7
8 #remove canceled out (~)-means to exclude
9 df = df[~df['InvoiceNo'].str.contains('C')]
10
11 df.shape
```

✓ [9] 3s 471ms

(531167, 8)

```
1 df.isnull().sum()
```

✓ [8] 213ms

InvoiceNo	0
StockCode	0
Description	0
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	133243
Country	0

dtype: int64

Phase 2 – Transaction Matrix Construction

UK transactions were reshaped into a binary transaction matrix: 18,668 invoices × 4,134 unique products. Each cell is 1 if the product appeared in that invoice, 0 otherwise.

Key decisions:

- Binarized quantities – we only care about the presence or absence of products, not how many units were bought.

- Removed non-product columns: postage, dotcom, carriage, manual – these are operational entries such as shipping costs and/or taxes.

Description	*Boombox Ipod Classic	*USB Office Mirror Ball	10 COLOUR SPACEBOY PEN	12 COLOURED PARTY BALLOONS	12 DAISY PEGS IN WOOD BOX	12 EGG HOUSE PAINTED WOOD	12 HANGING EGGS HAND PAINTED	12 IVORY ROSE PEG PLACE SETTINGS	12 MESSAGE CARDS WITH ENVELOPES	12 PENCIL SMALL TUBE WOODLAND	...	v
InvoiceNo												
536365	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
536366	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
536367	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
536368	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	
536369	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	

5 rows × 4134 columns

Phase 3 – Frequent Itemset Mining & Rule Generation

I applied the Apriori algorithm using the mlxtend Python library with:

- A minimum support of 2% — the itemset must appear in at least 2% of all UK transactions
- This produced 368 frequent itemset and 155 association rules.

During dashboard development, I noticed that a 2% threshold left too many products without recommendations. The threshold was lowered to 1.5% to ensure full product coverage. Because Apriori ran out of memory at the lower threshold on this dataset size, FP-Growth (an equivalent algorithm that uses a compressed tree structure instead of candidate generation) was used to regenerate the rules. FP-Growth produces identical rules to Apriori at the same threshold.

The final rule set used by the app:

- Minimum support: 1.5% — the itemset must appear in at least 1.5% of all UK transactions
- 692 frequent itemsets
- 2,795 association rules across 14 metrics (support, confidence, lift, leverage, conviction, and more)
- 241 products with full recommendation coverage

To filter for cross-sell, I ensured that the antecedent and consequent products' names share no significant words and exclude generic descriptors like "large," "small," and "design," to eliminate product variants.

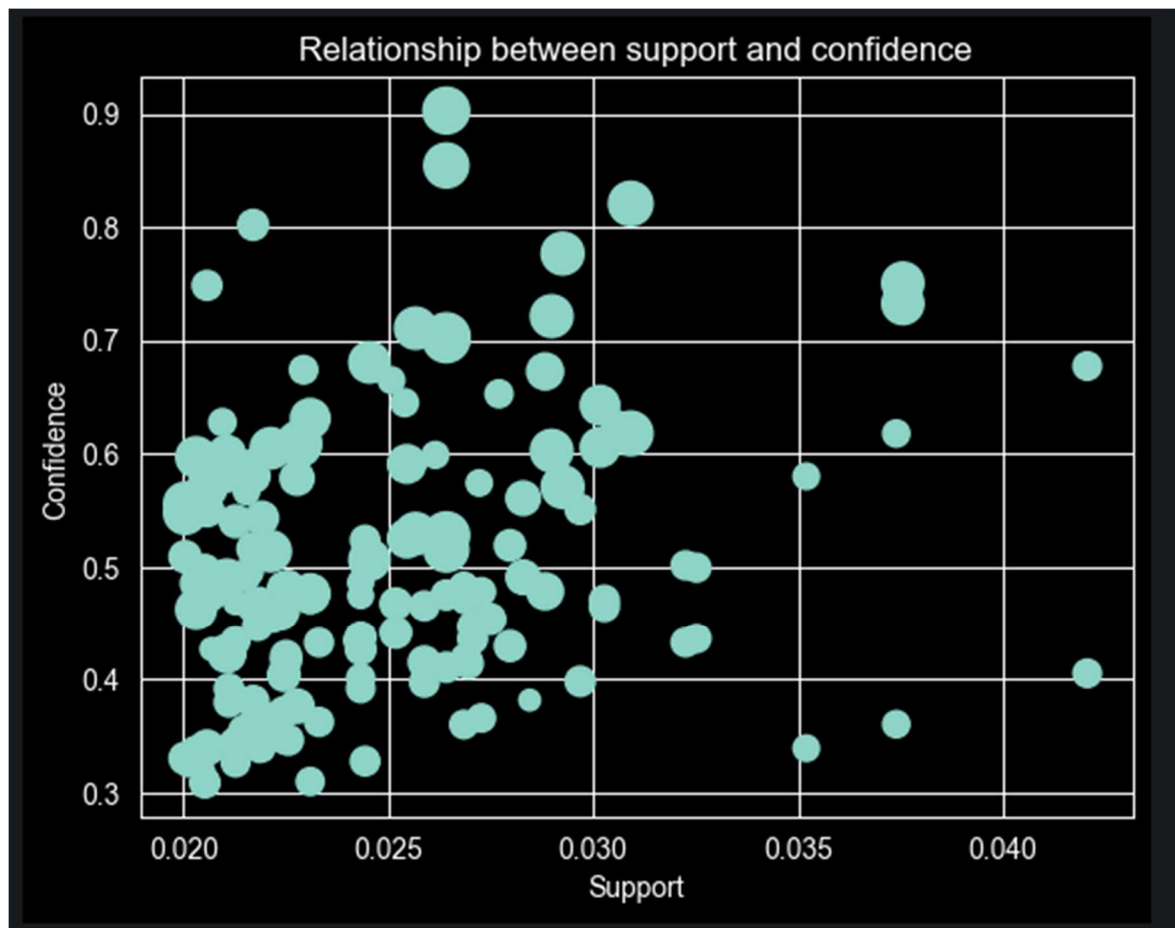
Strong rule:

	antecedents	consequents	support	confidence	lift
143	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...	0.026409	0.701280	18.675463
140	(GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...	(PINK REGENCY TEACUP AND SAUCER)	0.026409	0.703281	18.675463
141	(ROSES REGENCY TEACUP AND SAUCER, PINK REGENCY...	(GREEN REGENCY TEACUP AND SAUCER)	0.026409	0.902930	18.047007
142	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER, PINK REGENCY...	0.026409	0.527837	18.047007
139	(GREEN REGENCY TEACUP AND SAUCER, PINK REGENCY...	(ROSES REGENCY TEACUP AND SAUCER)	0.026409	0.854419	16.666982
144	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER, PINK REGENCY...	0.026409	0.515152	16.666982
23	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)	0.030909	0.820768	16.404818
22	(GREEN REGENCY TEACUP AND SAUCER)	(PINK REGENCY TEACUP AND SAUCER)	0.030909	0.617773	16.404818
18	(SPACEBOY LUNCH BOX)	(DOLLY GIRL LUNCH BOX)	0.022820	0.607703	16.229764
19	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)	0.022820	0.609442	16.229764

Phase 4 – Visualization

I visualized the relationship between support and confidence across all 155 rules, with bubble size representing lift strength. This scatter plot revealed clustering patterns – most high-confidence rules sit in the lower support range (2–3%), indicating strong but niche associations rather than globally common pairings.

Support vs. Confidence Scatter Plot:



Each bubble represents one association rule. Bubble size corresponds to lift, larger means the pairing is more statistically meaningful above random chance. High-confidence rules are visible in the upper-left cluster, representing strong but narrow associations.

Phase 5 – Dashboard Development

The analytical findings were deployed as an interactive web application using Streamlit. The app reads the exported rules.csv and provides a product dropdown that any user can interact with – no data science knowledge required.

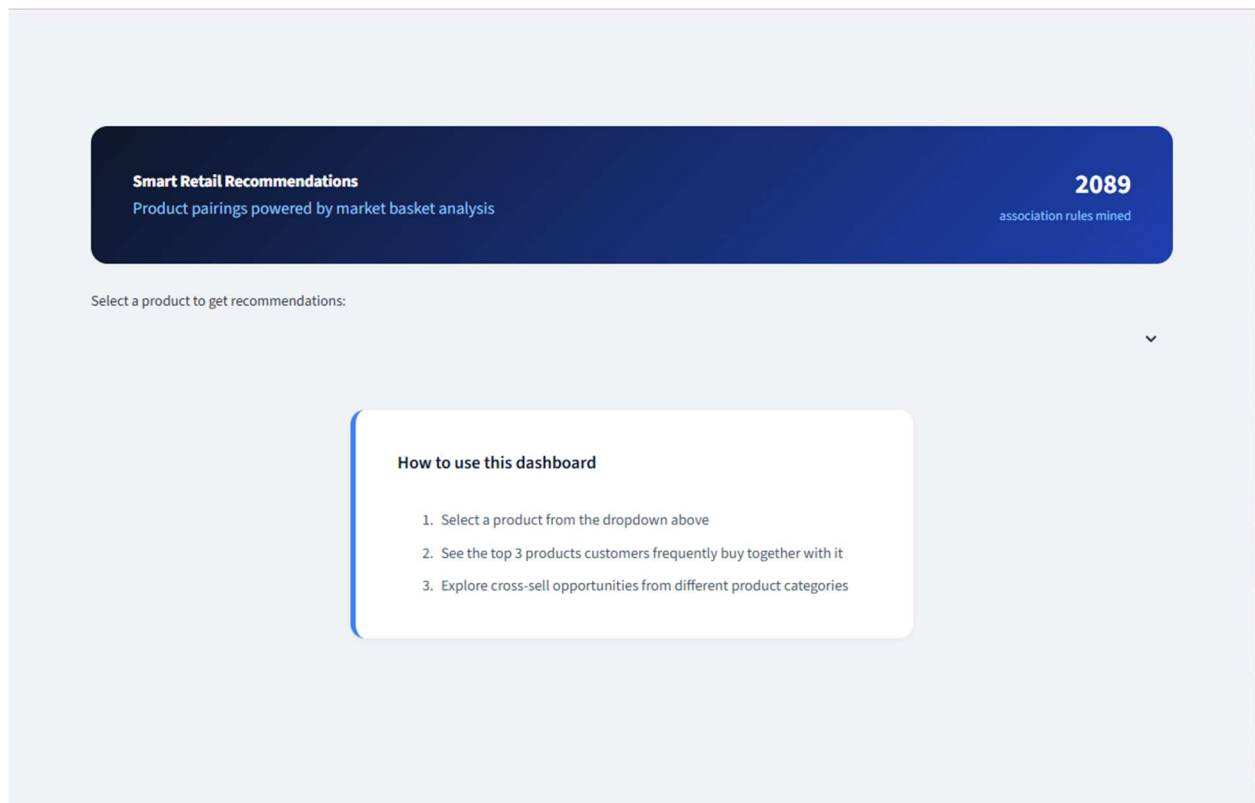
Technology Stack

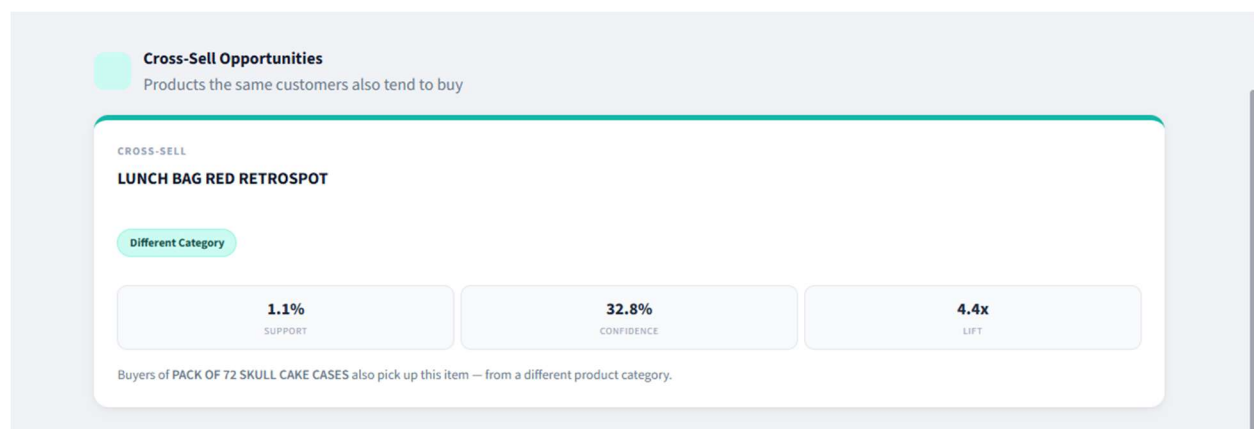
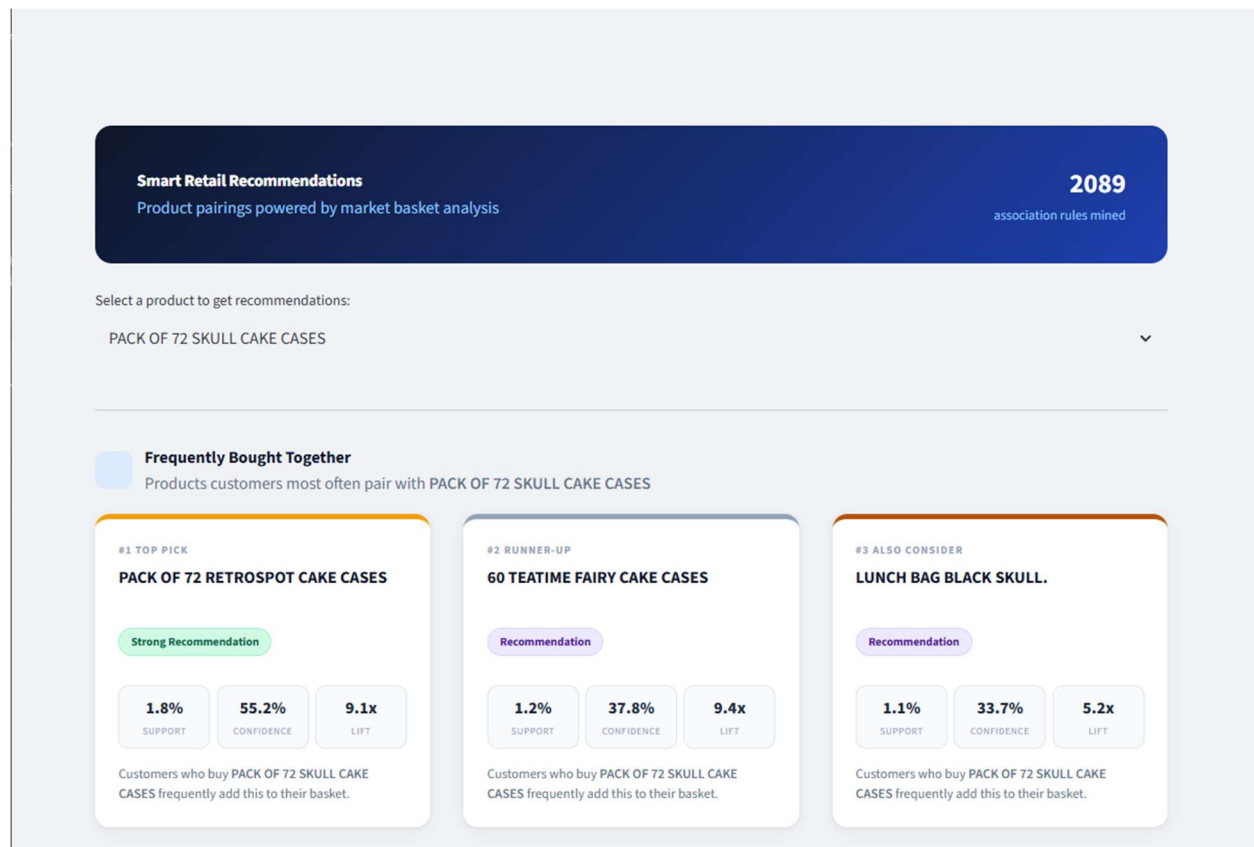
- Python 3.x
- Pandas – data manipulation
- mlxtend – Apriori algorithm and rule generation

- Streamlit – interactive web app framework
- Custom CSS – gradient hero banner, dark sidebar, card-based layout

Dashboard Architecture

- Sidebar: shows total products available and total rules mined, with threshold reference
- Hero banner: contextualizes the tool with a live rule count
- Section 1 — Frequently Bought Together: top 3 products by support + lift with gold/silver/bronze ranking and metric pills
- Section 2 — Cross-Sell Opportunities: top 3 cross-category products ranked by lift with a distinct teal card design
- Legend: explains support, confidence, and lift in plain English for business users





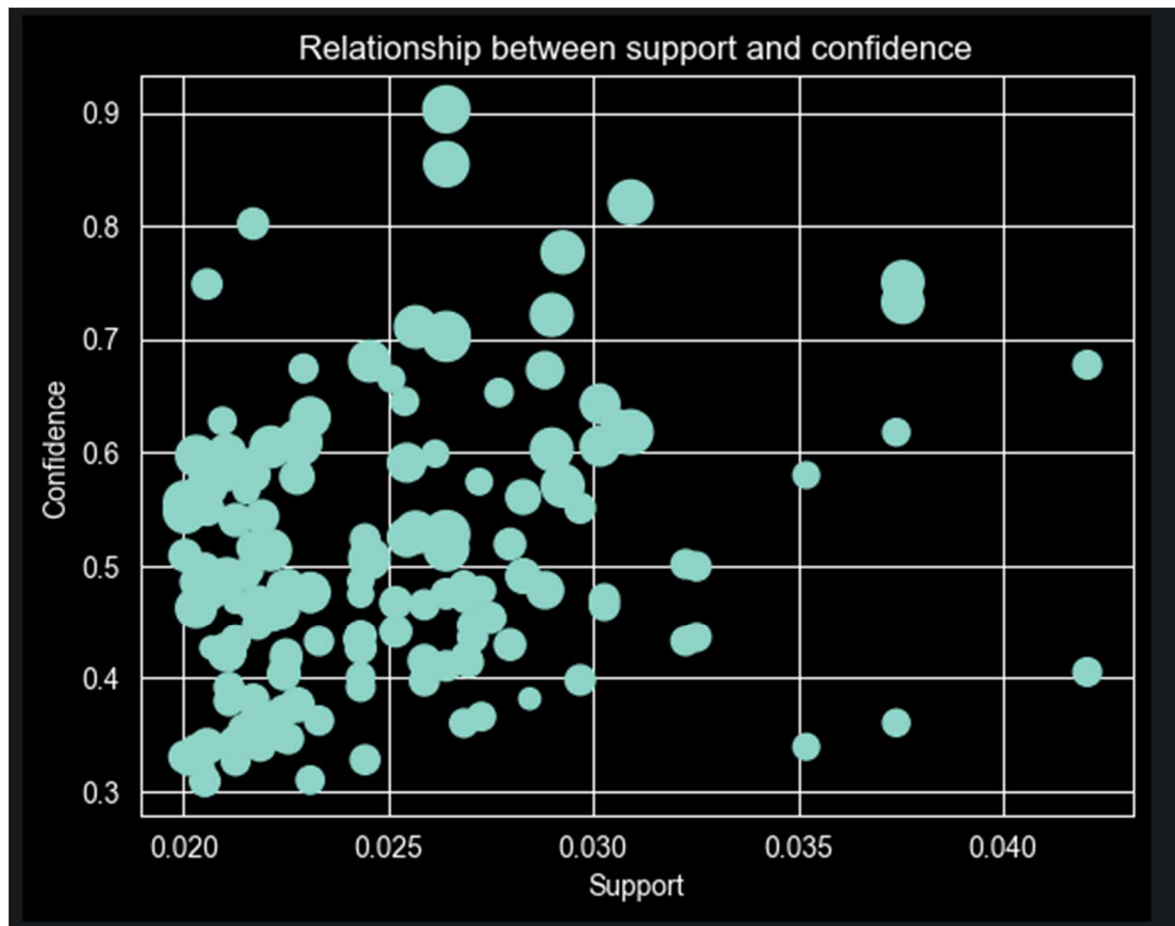
Highlight of the Project

Favorite Part 1 – The Association Rule Result

The strongest rule in the dataset told a genuinely interesting story. The top-lift rule 'PINK REGENCY TEACUP AND SAUCER → {GREEN REGENCY TEACUP AND SAUCER, ROSES REGENCY TEACUP AND SAUCER}' achieved a lift of 18.68. It means the pairing is over 18 times more likely than if they were purchased independently at random.

I found it fascinating how that translates behaviorally: customers aren't randomly picking teacups – they're deliberately completing a collection. That is a different kind of signal than a simple co-purchase. It revealed intentional, goal-directed shopping, which is a far richer insight than I expected from a simple algorithm.

The Charlotte bag results were equally revealing – customers building coordinated bag sets across the Woodland, Strawberry, Pink Polkadot, and Suki variants, with lifts ranging from 14.0 to 15.2.



This scatter plot was a key analytical step, it showed that the strongest rules (largest bubbles = highest lift) cluster in the moderate support range, not at the extremes. Rules with support above 0.035 and high confidence tend to be the collection-completion behaviors.

Favorite Part 2 – The Cross-Sell Detection Logic

Although I struggled with the `is_cross_sell()` function, I found it to be effective. By checking whether two product names share any significant keyword (ignoring short words and generic descriptors), it distinguishes:

- PINK REGENCY TEACUP → GREEN REGENCY TEACUP: same family (not cross-sell — both contain "REGENCY" and "TEACUP")
- JUMBO BAG PINK POLKADOT → JUMBO BAG RED RETROSPOT: within-family pairing (shares "JUMBO" and "BAG")

The heuristic isn't perfect, but for a keyword-rich product catalog like this one, it draws a meaningful and useful line.

Conclusion

What I Learned

Interpreting association rule mining: support, confidence, and lift are three numbers, but they hold meaningful behavioral signals. Learning to read them together, not just pick the highest lift, taught me how to translate statistical output into business insight.

Data cleaning: the decisions made before any model runs are which rows to drop, how to normalize, and which values to exclude. For example, choosing to binarize quantities rather than preserve counts was not an obvious call. Getting comfortable with those judgment calls was one of the most valuable skills I developed.

Presentation is part of analytics: a recommendation engine that only a data scientist can query isn't a recommendation engine, it's a research tool. Building the Streamlit dashboard forced me to think about what business users like store managers need to see and how to communicate statistical metrics without statistical jargon.

AI-assisted development is a real workflow: collaborating with Claude on the dashboard's UI architecture, CSS system, and cross-sell logic was genuinely productive. It shifted time away from implementation details toward higher-level decisions about what the tool should do and why.

What I Did Well

The full pipeline: I took this from a raw CSV to a deployed, interactive web application. That's the complete data product lifecycle — understand the data, clean it, model it, interpret the results, and make them accessible.

Keeping the cleaning conservative: I only removed records that genuinely couldn't contribute to basket analysis. Aggressive cleaning would have shrunk the dataset and potentially hidden real purchasing patterns.

The two-tier recommendation architecture: separating "frequently bought together" from "cross-sell opportunities" wasn't the obvious first design. It required recognizing that bundle promotion and category expansion serve different business objectives and should be surfaced differently.

What I Could Have Done

- Seasonal insights: the dataset spans a full year. Splitting by season (e.g., Q4 holiday vs. Q1 off-peak) would likely reveal very different association patterns and enable seasonal recommendation tuning.
- The support threshold: I initially settled at 2% due to memory constraints, but it left too many products without recommendations on the dashboard. Lowering 1.5% resolved the coverage gap and expanded the rules from 155 to 2,795 across 241 products. Going further to 0.5% or lower might surface meaningful niche patterns but would require a more memory-efficient implementation pipeline.
- Visualizations: the scatter plot I produced is informative but minimal. A lift network graph showing which products cluster together, or a heatmap of the strongest rule pairs, would have communicated the findings more powerfully.

Main Takeaway

The most useful thing I will carry forward from this project is how to move from analysis to artifact — not stopping at findings but continuing until those findings are in a form that someone else can use.

Data science is too often treated as ending at the results table. This project made it concrete that the results table is the middle, not the end. The recommendation engine is only valuable because someone who has never opened a Jupyter Notebook can pick a product from a dropdown and immediately see actionable intelligence. The real work is building the bridge between what the data says and what a business can do with it.